Task 02 - LLM, Context and Memory: the Context Packet

Handbook reference: Page 2 of 10 | Report date: 2026-10-13 | Folder: `task-02-llm-context-memory/`

1. Concept from the handbook

An LLM works only on the context it is given (prompt, tokens, a limited context window). Missing API contracts, tests or repo instructions lead to plausible but wrong code. Saved memory is not an automatically current repository record - the source code and repository instructions stay the source of truth. Give **relevant, current, bounded** context, never secrets.

2. What was built

A 6-field **ContextPacket** (Goal, Files, Current, Expected, Constraints, Acceptance criteria), a **gate** that rejects incomplete packets or packets containing secrets, and a **file selector** that ranks files by relevance, stays inside a token budget, always includes AGENTS.md, and skips files that contain secrets.

3. Behaviours demonstrated

- Missing fields are reported before the agent starts
- API keys / passwords / e-mails detected by regex and blocked
- Relevant files chosen under a token budget (~4 chars per token)
- 'List your assumptions first' prompt included

4. Files in this task

`context_packet.py`, `demo.py`, `sample_repo/AGENTS.md`, `sample_repo/auth.py`, `sample_repo/billing.py`, `sample_repo/task_list.py`, `sample_repo/test_task_list.py`, `test_context_packet.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

context_packet.py

```python
"""Task 02 - Context packet: relevant, current, bounded information.

A 6-field packet (Goal, Files, Current, Expected, Constraints, Acceptance)
plus guards: secret scanning and a token budget for file selection.
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field, fields
from pathlib import Path
from typing import Dict, List

FIELDS = ["goal", "files", "current", "expected", "constraints", "acceptance"]

SECRET_PATTERNS = {
    "api key": re.compile(r"sk-[A-Za-z0-9_-]{16,}"),
    "aws key": re.compile(r"AKIA[0-9A-Z]{16}"),
    "password assignment": re.compile(r"(?i)\b(password|passwd|pwd|secret|token)\b\s*[=:]\s*['\"][^'\"]{4,}['\"]"),
    "private key": re.compile(r"-----BEGIN [A-Z ]*PRIVATE KEY-----"),
    "email (possible customer data)": re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+"),
}


@dataclass
class ContextPacket:
    goal: str = ""
    files: List[str] = field(default_factory=list)
    current: str = ""
    expected: str = ""
    constraints: str = ""
    acceptance: str = ""

    def missing(self) -> List[str]:
        return [f for f in FIELDS if not getattr(self, f)]

    def render(self) -> str:
        files = ", ".join(self.files) or "-"
        return "\n".join([
            f"Goal: {self.goal}",
            f"Files: {files}",
            f"Current: {self.current}",
            f"Expected: {self.expected}",
            f"Constraints: {self.constraints}",
            f"Acceptance criteria: {self.acceptance}",
        ])


def scan_secrets(text: str) -> List[str]:
    return [name for name, rx in SECRET_PATTERNS.items() if rx.search(text)]


def estimate_tokens(text: str) -> int:
    """Rough rule of thumb: ~4 characters per token."""
    return max(1, len(text) // 4)


def select_relevant_files(repo: Path, keywords: List[str], budget_tokens: int = 400) -> Dict[str, int]:
    """Rank files by keyword hits; include best ones until the budget is used.

    Files that contain secrets are skipped entirely (never put in context).
    Returns {relative_path: estimated_tokens}.
    """
    scored = []
    for p in sorted(Path(repo).rglob("*")):
        if not p.is_file() or p.suffix not in {".py", ".md"}:
            continue
        text = p.read_text()
        if scan_secrets(text):
            continue
        hits = sum(text.lower().count(k.lower()) for k in keywords)
        if p.name == "AGENTS.md":          # repository instructions always matter
            hits += 100
        if hits:
            scored.append((hits, p, estimate_tokens(text)))
    chosen, used = {}, 0
    for hits, p, tok in sorted(scored, key=lambda t: -t[0]):
        if used + tok > budget_tokens:
            continue
        chosen[str(p.relative_to(repo))] = tok
        used += tok
```

Main implementation: context_packet.py (part 1/2)

```
    return chosen


def prepare(packet: ContextPacket) -> dict:
    """Gate: refuse to send a packet that is incomplete or leaks secrets."""
    problems = []
    if packet.missing():
        problems.append(f"missing fields: {', '.join(packet.missing())}")
    leaks = scan_secrets(packet.render())
    if leaks:
        problems.append(f"possible secrets in context: {', '.join(leaks)}")
    return {"ok": not problems, "problems": problems,
            "tokens": estimate_tokens(packet.render())}


ASSUMPTION_PROMPT = ("Before coding, list every assumption you are making and anything "
                     "in this packet that is missing or ambiguous.")
```

Main implementation: context_packet.py (part 2/2)

demo.py

```
from pathlib import Path

from context_packet import ASSUMPTION_PROMPT, ContextPacket, prepare, select_relevant_files

repo = Path(__file__).parent / "sample_repo"

print("1) Which files are relevant? (bounded budget, secrets skipped)")
chosen = select_relevant_files(repo, ["task", "filter", "status"], budget_tokens=400)
for f, t in chosen.items():
    print(f"   + {f:16s} ~{t} tokens")
print("   - billing.py  (irrelevant)   - auth.py  (contains a secret)\n")

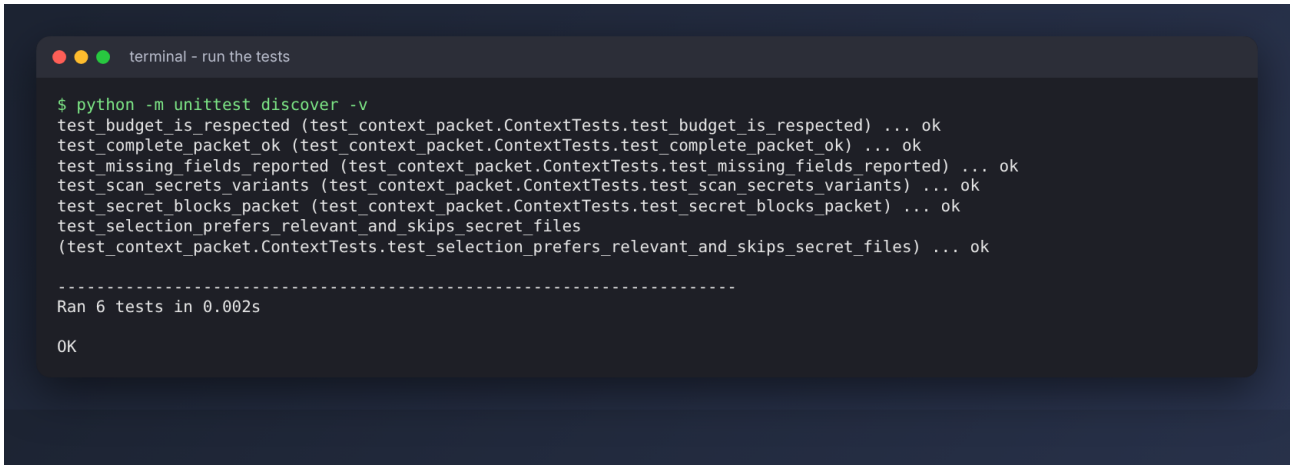
print("2) The 6-line context packet")
packet = ContextPacket(
    goal="Add a status filter (All / Open / Done) to Task Tracker",
    files=list(chosen),
    current="All tasks are always shown",
    expected="Only tasks with the selected status are shown",
    constraints="Existing stack, stdlib only; no unrelated refactor",
    acceptance="All/Open/Done cases pass; current behaviour preserved",
)
print(packet.render(), "\n")
print("   gate ->", prepare(packet), "\n")

print("3) A bad packet: missing acceptance + leaked key")
bad = ContextPacket(goal="fix filter", files=["auth.py"], current="API_KEY=sk-demo-FAKEKEY0123456789abcdef",
                    expected="works", constraints="none")
print("   gate ->", prepare(bad), "\n")

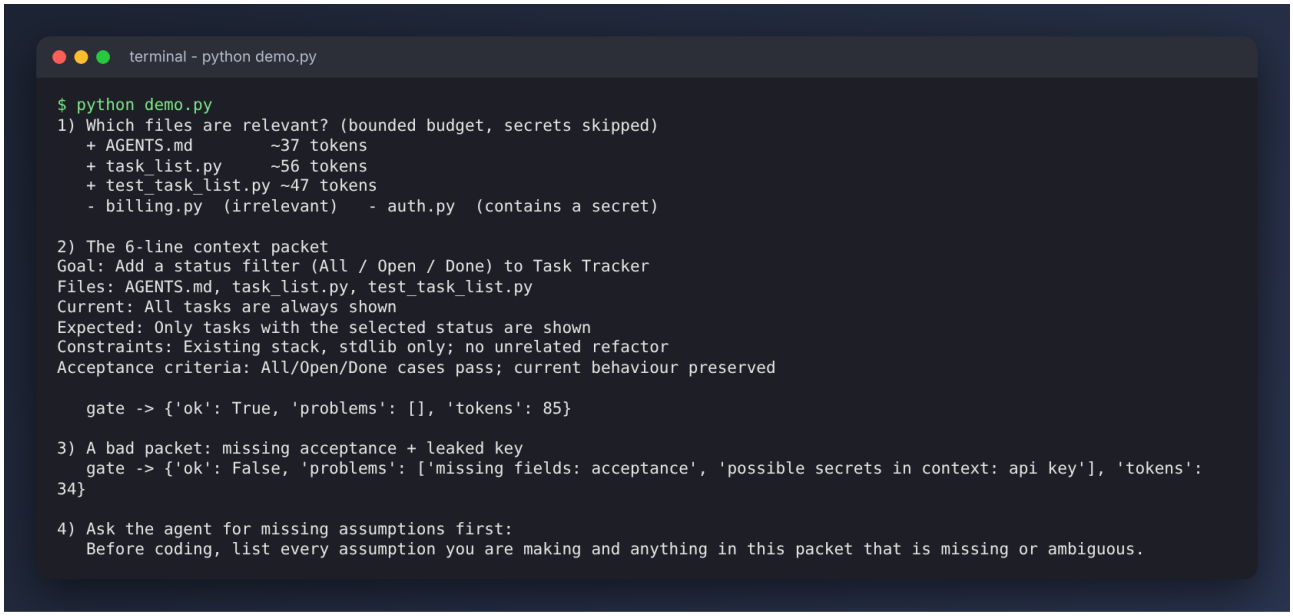
print("4) Ask the agent for missing assumptions first:")
print("  ", ASSUMPTION_PROMPT)
```

Demo script

6. Demo and test screenshots



Test run (real output)



Demo output (python demo.py)

7. Evidence-backed report

Change	LLM, Context and Memory: the Context Packet
Revision tested	content hash `8b683758b6fd` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 6 tests in 0.003s
Checks not run	Real tokenizer (token count is an estimate)
Known limitations	Secret detection is regex-based and will miss unusual formats; it is a safety net, not a guarantee.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

Strong context means relevant, current and bounded information - not more text.